Terraform State Management & Environment Separation

WEEK 2

Learning Objectives

- Explain what Terraform state is and why it exists
- Read and interpret a terraform.tfstate file
- Configure remote backends
- Use Workspaces and file-layout isolation for multiple environments
- Apply best practices for production-grade infrastructure

Outline

- What is Terraform State?
- State File Structure & Inspection
- Local vs Remote State + Backends
- **LAB 1 — Configure S3 Remote Backend**
- Terraform Workspaces
- File Layout Isolation (Recommended Pattern)
- **LAB 2 — Multi-Environment Setup (dev/staging/prod)**
- Review, Q&A, and Best Practices Cheat Sheet

Terraform State

- Terraform state acts as Terraforms memory.
- It records a mapping between the resources defined in your configuration and the real infrastructure that exists in your cloud provider.
- By default, when you run Terraform in the folder /labs/labs1b, Terraform creates a file /labs/labs1b/terraform.tfstate.
- The state file ,terraform.tfstate is a customed JSON file
- The state file formats a private API t at is meant only for internal use within Terraform.
- You should never edit the Terraform state files by and or write code that reads them directly.

Why Does State Exist?

Without State

You run terraform apply

Terraform creates EC2 instance i-abc123

You run terraform apply again...

Terraform has NO memory of i-abc123

It tries to create ANOTHER EC2 instance!

You now have duplicate resources

Cloud bill doubles. On-call gets paged.

Why Does State Exist?

WITH STATE

You run terraform apply

Terraform creates EC2 instance i-abc123

State file records: i-abc123 exists

You run terraform apply again...

Terraform checks state file

Sees i-abc123 already exists

No changes. Infrastructure is up-to-date

What Terraform State Does :4 Key Roles

1. **Resource Mapping**

   Links your configs (aws_instance.web) to the real resources (i-0abc123def456) .Without this, Terraform cannot manage existing resources.

2. **Dependency Tracking**

   Records which resource depends on which. A security group must exist before an EC2 instance uses it. State tracks this graph.

3. **Performance Optimization**

   Terraform doesn't call the cloud API for every resource on every run. It reads state first. Much faster for large infra with 100s of resources.

4. **Metadata Storage**

   Stores provider-specific data, resource schema versions, and other info needed for safe upgrades and migrations.

Anatomy of the State File

- **`version:`** Indicates the state file format version (schema version) , it is always 4 in modern Terraform.
- **terraform version :** Specifies the Terraform version used to create this state file.
- **`serial:`** An integer that increments with every state change, used for tracking changes and ensuring safe updates (especially with remote backends).
- **`outputs:`** Output values from your config, readable by other modules
- **`resources:`** The core section — every real cloud resource Terraform manages, with all its attributes.

Inside terraform.tfstate — What It Looks Like


```
{
  "version": 4,
  "terraform_version": "1.7.0",
  "serial": 12,
  "lineage": "3b4d...uuid",
  "outputs": {
    "instance_ip": {
      "value": "54.12.34.56",
      "type": "string"
    }
  },
  "resources": [
    {
      "mode": "managed",
      "type": "aws_instance",
      "name": "web_server",
      "provider": "registry...",
      "instances": [{
        "schema_version": 1,
        "attributes": {
```

Inspecting State

- **terraform show:** Print the full state in human-readable form

Example

```
resource "aws_instance" "web_server" {
  id          = "i-0abc1234def56789"
  instance_type = "t3.micro"
  public_ip    = "54.12.34.56"
}
```

- **terraform state list:** List all resources tracked in state

Example

```
aws_instance.web_server
aws_security_group.web_sg
aws_s3_bucket.static_assets
```

Inspecting State

- **terraform state show aws_instance.web_server:** Show details of a single resource

```
id          = "i-0abc1234"
ami         = "ami-0c55b159"
tags        = { Name = "web-server" }
```

State Backup File

- Every time you run **terraform apply** or **terraform destroy**, Terraform saves the PREVIOUS state as **terraform.tfstate.backup** before overwriting the current state.
- **One version back only**
  - Only the immediately previous state is kept. It is overwritten on the next operation.
- **Recovery lifeline**
  - If a terraform apply partially fails or corrupts state, the backup lets you restore to the last known-good state.
- **Not a long-term backup**
  - For real protection, use versioned S3 buckets or other remote backends with versioning enabled.

Terraform Local State

- Local state is the default method Terraform uses to store your infrastructure's "source of truth" as a terraform.tfstate file on your individual machine.

When to Use Local State

- **Learning & Prototyping:** Ideal for following a Terraform Tutorial or exploring new providers.
- **Quick Sanity Checks:** Testing small code blocks locally before committing to a shared repo.
- **Single-Developer Projects:** Personal hobby projects where you are the only one managing the infrastructure.

Terraform Local State

Why Local State is Unsuitable for Teams

- **Shared storage for state files**

  To be ab le to use Terraform to update your infrastructure, each of your team members needs access to the same Terraform state files. That means you need to store those files in a shared location which local state does not provide.

- **No State Locking**

  Without state locking, if two team members are running Terraform at the same time, you can run into race conditions as multiple Terraform processes make concurrent updates to the state files, leading to conflicts, data loss, and state file corruption.

- **Accidental Deletion & Loss**

  Because the state file lives on a local laptop or machine, it is fragile. If the machine crashes, the disk is wiped, or a user accidentally runs rm -rf terraform.tfstate, the team loses the map of the infrastructure.

Terraform State: Shared storage for state files

- Version control e.g., Git allows teams to work together on a common file.
- Terraform *code (config)* should be version-controlled.
- Storing Terraform state files in version control (e.g., Git) anti-pattern in DevOps for these reasons:
  1. Security Risks (Secrets):

     All resource details are stored in plain text in state files.
  2. Lack of State Locking

     Git is not designed for the synchronous nature of infrastructure changes.
  3. Manual Error

     Relying on manual commits leads to "State Drift.

Terraform Remote State

- The best way to manage shared storage for state files is to use Terraforms bult-in support for remote backends.
- A Terraform backend determines how Terraform loads and stores state.
- **Terraform support a number of remote backends, including;**
  - Amazon S3
  - **Azure Blob**
  - **Google Cloud Storage**
  - **Terraform Cloud**
- Limitations wit Terraform's Backends.
- The first limitation is the chicken-and-egg situation of using Terraform to create the S3 bucket where you want to store your Terraform state.

Choosing a Remote Backend

Feature	AWS S3	Google Cloud Storage	Azure Blob	Terraform Cloud
Storage	S3 Bucket	GCS Bucket	Storage Account	Managed
State Locking	Use to be DynamoDB table but now is Built-in	Built-in (native)	Built-in (native)	Built-in (native)
Encryption	SSE-S3 / KMS	Google-managed / CMEK	Azure-managed / BYOK	Built-in
Versioning	S3 Versioning	Object Versioning	Blob Versioning	Built-in
Cost	Pay per request	Pay per request	Pay per request	Free tier available

LAB 2A: Configure Remote State with S3 + DynamoDB Locking

- In this Lab we will Configure Terraform to use a remote backend by creating an Amazon S3 bucket for storing Terraform state files and a DynamoDB table for state locking.

Workspaces and file-layout isolation for multiple environments

Terraform Workspaces

- Terraform workspaces allow you to store your Terraform state n multiple, separate, named workspaces.
- Terraform workspaces allow you to manage multiple isolated instances of your infrastructure state within the same configuration directory.
- Terraform starts with a single, default workspace named **default** that you cannot delete.
- Instead of duplicating your code into separate folders for different environments (like dev, staging, and prod), you can use one set of code and switch between different state files.
- *Think of workspaces like Git branches — same code, different state.*

Workspace Commands

- `terraform workspace list:`
- `terraform workspace new dev:`
- `terraform workspace select prod:`
- `terraform workspace show:`

Workspace Limitations

- All environments share the same backend bucket
- Harder to have different provider credentials per env
- Easy to accidentally apply to wrong workspace
- Not recommended for strong prod isolation

Workspaces Work Well For

- Feature branch environments
- Quick test environments
- Simple projects with 1-2 engineers
- Learning and experimentation

File Layout Isolation — The Recommended Pattern

- *The gold standard for production infra: each environment is a completely independent Terraform root module with its own state.*
- **Blast Radius Control**
- A mistake in dev CANNOT affect prod. Different state files, different workspaces, often different AWS accounts.
- **Different Credentials**
- Each environment directory can use a different AWS profile or IAM role. Prod requires MFA; dev doesn't.
- **Clear Review Process**
- A PR touching environments/prod/ is immediately obvious. Reviewers know to be extra careful.
- **Per-env Variables**
- terraform.tfvars in each directory sets env-specific values: instance sizes, replica counts, domain names.

File Layout Isolation — Separate Directories Per Environment

```
infra/
├── modules/
│   ├── networking/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   └── compute/
│       ├── main.tf
│       └── variables.tf
│
├── environments/
│   ├── dev/
│   │   ├── main.tf          ← calls modules
│   │   ├── backend.tf       ← dev state
│   │   └── terraform.tfvars
│   ├── staging/
│   │   ├── main.tf
│   │   ├── backend.tf       ← staging state
│   │   └── terraform.tfvars
│   └── prod/
│       ├── main.tf
│       ├── backend.tf       ← prod state
│       └── terraform.tfvars
```

Best Practices — Production Terraform State Cheat Sheet

- Always Enable State Locking
- Use Remote State from Day 1
- Version Your State
- Never Commit State to Git
- Use File Layout for Prod Envs
- Inspect Before You Destroy

Quick Reference — State Management Commands

Inspect State

```
$ terraform show
$ terraform state list
$ terraform state show <resource>
$ terraform output
```

Modify State (Advanced)

```
$ terraform state mv <src> <dst>   # rename
$ terraform state rm <resource>     # stop tracking
$ terraform import <resource> <id>  # adopt existing
```

Workspaces

```
$ terraform workspace list
$ terraform workspace new <name>
$ terraform workspace select <name>
$ terraform workspace delete <name>
```

Backend / Init

```
$ terraform init                # first time
$ terraform init -reconfigure # new backend
$ terraform init -migrate-state
```

Key Takeaways

State is Terraform's memory — it maps config to real cloud resources. Without it, Terraform is blind.

Always use remote state for any shared or production infrastructure. S3+DynamoDB is the AWS standard.

State locking prevents corruption. Two applies running at once without a lock = corrupted state file.

Use Workspaces for lightweight env separation. Use File Layout for true prod-grade isolation.

Never commit state files to Git. Enable S3 versioning. Treat state with the same care as your database.

Next Week: Terraform Modules & Reusable Infrastructure Patterns